

Leaky Bucket Algorithm (Rate Limiting)

The **Leaky Bucket Algorithm** is a rate limiting technique that processes requests at a **fixed, steady rate**.

It is similar to the Token Bucket algorithm, but instead of allowing burst processing, it smooths traffic output.

Used by:

- Shopify

1. Core Idea

Imagine a bucket with a small hole at the bottom:

None

Requests enter bucket (queue)

Requests leave bucket at constant speed

If incoming traffic is faster than outflow, the bucket fills up.

If bucket becomes full, new requests are rejected.

2. How It Works

Step 1: Incoming Request Arrives

System checks whether queue has space.

None

Queue not full → add request

Queue full → reject/drop request

Step 2: Fixed Processing Rate

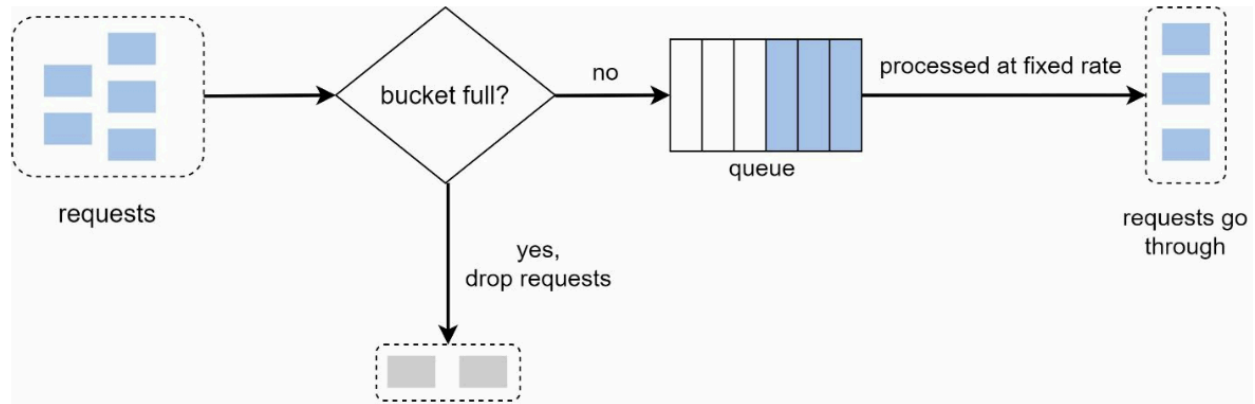
Requests are processed from queue at regular intervals.

None

FIFO queue:

Oldest request processed first

Step 3: Continuous Drain



As shown in **Figure**:

None

Queue receives requests

Processor drains queue steadily

3. Two Important Parameters

Bucket Size

Maximum queue capacity.

None

Controls how many waiting requests can be stored

Larger queue = can absorb larger spikes.

Outflow Rate

How many requests are processed per second/minute.

None

Controls stable throughput

Example:

None

10 requests / second

4. Example Flow

None

Bucket size = 5

Outflow rate = 1 req/sec

Second 1:

4 requests arrive → queue stores 4

Second 2:

3 more arrive

1 processed

Queue can hold only 2 more

1 request dropped

5. Why It Is Useful

✓ Smooth Traffic

Even if incoming traffic is bursty:

None

Output remains stable and predictable

Useful for downstream systems that cannot handle spikes.

✓ **Protects Backend Services**

Database or worker systems receive controlled request flow.

✓ **Memory Efficient**

Queue has fixed size.

6. Advantages

- Easy to understand
- Memory efficient (bounded queue)
- Stable output rate
- Good for systems needing predictable throughput
- Prevents sudden overload on downstream systems

7. Disadvantages

Old Requests Fill Queue

During bursts:

None

Older requests occupy queue

Newer requests may be rejected

Poor Burst Handling

Unlike Token Bucket, it does not naturally allow instant bursts.

Parameter Tuning Needed

Need correct:

- Queue size
- Outflow rate

8. Token Bucket vs Leaky Bucket

Feature	Token Bucket	Leaky Bucket
Burst Traffic	Allowed	Limited
Output Rate	Variable	Fixed
Queue Needed	Not necessary	Yes

Best For	APIs / bursts	Stable downstream flow
----------	---------------	------------------------

9. Real-World Use Cases

- Payment processing pipelines
- Email sending queues
- Job worker throttling
- Network traffic shaping
- Backend systems needing constant load

10. Interview Deep Dive Points

Implementation options:

- In-memory queue
- Redis list/stream queue
- Distributed workers consuming at fixed rate

Consider:

- Queue timeout
- Retry strategy
- Priority queues

Mental Model

None

Leaky Bucket = Buffer burst traffic + release steadily

One-Line Summary

Leaky Bucket Algorithm rate-limits traffic by placing requests in a fixed-size queue and processing them at a constant rate, making it ideal for systems that need smooth and predictable throughput.